

BÀI THỰC HÀNH

LAB 07

Cơ bản về Tone

Tạo âm thanh với Passive Buzzer bằng EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	Passive Buzzer và tín hiệu điều khiển	2
2.2	Chu kỳ và tần số	2
2.3	Tần số và cao độ	3
2.4	Tần số và thời lượng của một tone	3
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Kết nối Passive Buzzer	3
4	EduFramework API	4
4.1	<code>tone()</code>	4
4.2	<code>noTone()</code>	5
4.3	Lưu ý về tài nguyên PWM	5
5	Bài thực hành	5
5.1	Yêu cầu	5
5.2	Chương trình	6
5.3	Kiểm chứng	6
6	Mở rộng	7
6.1	Đặt tên tần số bằng hằng số	7
6.2	Bài tập mở rộng: tạo một giai điệu ngắn	7
7	Tài liệu tham khảo	7

1 Giới thiệu

1.1 Tổng quan bài lab

Âm thanh thường được sử dụng trong hệ thống nhúng để báo trạng thái, phát cảnh báo hoặc cung cấp phản hồi cho người sử dụng. Với passive buzzer, âm thanh có thể được tạo bằng một tín hiệu tuần hoàn; thay đổi tần số của tín hiệu làm thay đổi cao độ nghe được [2] [3] .

Bài lab này giới thiệu cách tạo các tone đơn giản bằng EduFramework. Passive buzzer được kết nối với GPIO7 , sau đó chương trình lần lượt phát các tần số khác nhau để tạo một chuỗi âm có cao độ tăng dần. Nội dung tập trung vào ba yếu tố chính: tín hiệu tuần hoàn, quan hệ giữa tần số và cao độ, và cách điều khiển buzzer bằng API của framework.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Giải thích nguyên lý cơ bản của việc tạo âm thanh bằng passive buzzer.
2. Mô tả mối quan hệ giữa chu kỳ, tần số và cao độ của âm thanh.
3. Sử dụng `tone()` và `noTone()` để điều khiển tone bằng EduFramework.
4. Xây dựng và kiểm chứng chương trình phát một chuỗi âm liên tiếp trên passive buzzer.

2 Kiến thức nền

2.1 Passive Buzzer và tín hiệu điều khiển

Passive buzzer không tự tạo một tone cố định khi được cấp một mức logic không đổi. Để phát âm, buzzer cần được kích bằng một tín hiệu thay đổi tuần hoàn theo thời gian. Trong các bài thực hành hệ thống nhúng, tín hiệu vuông là một cách phổ biến để tạo tín hiệu kích này [2] [3] .

Tín hiệu vuông luân phiên giữa hai mức logic. Khi quá trình chuyển mức được lặp lại đều đặn, tín hiệu có một tần số xác định. Bằng cách thay đổi tần số điều khiển, cùng một buzzer có thể phát ra các tone có cao độ khác nhau [2] .

Trong phạm vi bài lab, buzzer được sử dụng như một thiết bị phát âm đơn giản. Các đặc tính chuyên sâu như đáp tuyến tần số, mức áp suất âm hoặc mô hình điện của phần tử buzzer không được xét đến.

2.2 Chu kỳ và tần số

Một tín hiệu tuần hoàn lặp lại sau một khoảng thời gian xác định. Khoảng thời gian để hoàn thành một lần lặp được gọi là chu kỳ và thường ký hiệu là T . Tần số cho biết số chu kỳ xảy ra trong một giây và được biểu diễn bằng đơn vị hertz (Hz) [1] .

Đối với một tín hiệu tuần hoàn, tần số và chu kỳ có quan hệ nghịch đảo:

$$f = \frac{1}{T}$$

Trong đó:

- f là tần số, đơn vị hertz.
- T là chu kỳ, đơn vị giây.

Vì vậy, tần số càng lớn thì thời gian của một chu kỳ càng ngắn. Ví dụ, một tín hiệu 500 Hz lặp lại nhanh hơn một tín hiệu 250 Hz, nên khoảng thời gian giữa các chu kỳ của tín hiệu 500 Hz nhỏ hơn.

2.3 Tần số và cao độ

Cao độ là cảm nhận cho biết một âm nghe cao hay thấp. Đối với các tone tuần hoàn đơn giản, cao độ có quan hệ chặt chẽ với tần số: tần số tăng làm cao độ cảm nhận tăng, còn tần số giảm làm cao độ cảm nhận giảm [1]. Quan hệ này là cơ sở của bài thực hành. Khi buzzer lần lượt nhận các tín hiệu có tần số tăng dần, chuỗi âm nghe được cũng tăng dần về cao độ [2].

Tần số	Chu kỳ	Cao độ cảm nhận
Thấp hơn	Dài hơn	Âm có cao độ thấp hơn.
Cao hơn	Ngắn hơn	Âm có cao độ cao hơn.

Bảng 7.2.1. Quan hệ định tính giữa tần số, chu kỳ và cao độ

2.4 Tần số và thời lượng của một tone

Tần số xác định cao độ của tone, còn thời lượng xác định tone được duy trì trong bao lâu. Hai đại lượng này có vai trò khác nhau: một tone có thể giữ nguyên tần số nhưng được phát trong thời gian ngắn hoặc dài tùy yêu cầu của ứng dụng. Các bài thực hành với buzzer thường mô tả riêng tần số theo hertz và thời lượng theo millisecond [3].

Trong Lab 07, mỗi tone được giữ trong 500 ms. Sau chuỗi tone, buzzer được dừng trong 1000 ms để tạo khoảng im lặng trước khi chuỗi bắt đầu lại.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng MaaZEDU Development Board với vi điều khiển S32K144 và một passive buzzer bên ngoài [7].

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
Passive Buzzer	Thiết bị phát âm được điều khiển bằng tín hiệu tuần hoàn.
Jumper wires	Kết nối buzzer với board.
USB Cable	Kết nối board với máy tính để cấp nguồn và nạp chương trình.

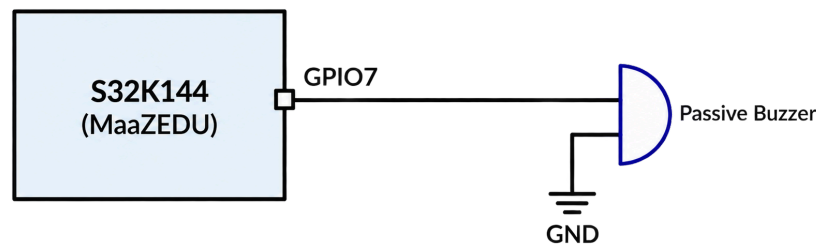
Bảng 7.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Kết nối Passive Buzzer

GPIO7 là một Logical Pin có khả năng PWM trong EduFramework và được sử dụng làm đầu ra điều khiển buzzer trong bài thực hành [6].

Thiết bị	Kết nối	Chức năng
Passive Buzzer	GPIO7	Nhận tín hiệu điều khiển để tạo âm thanh.
Passive Buzzer	GND	Nối về GND của MaaZEDU Development Board.

Bảng 7.3.2. Kết nối Passive Buzzer trong Lab 07



Hình 7.3.1. Sơ đồ kết nối Passive Buzzer với GPIO7

4 EduFramework API

Arduino Language Reference sử dụng `tone()` và `noTone()` cho nhóm thao tác tạo và dừng tone [4]. EduFramework cung cấp hai API cùng tên với giao diện phù hợp cho MaaZEDU.

4.1 `tone()`

`tone()` bắt đầu tạo tín hiệu vuông với tần số được chỉ định trên một Logical Pin có khả năng PWM. Trong implementation hiện tại, tín hiệu được tạo với duty cycle 50% [6].

`tone`

Bắt đầu phát tone ở tần số được chỉ định trên một PWM Logical Pin.

Cú pháp

```
tone(pin, frequency);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>pin</code>	PWM Logical Pin	Chân được sử dụng để điều khiển buzzer, ví dụ <code>GPIO7</code> .
<code>frequency</code>	Tần số (Hz)	Tần số của tone cần phát, tính bằng hertz.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
tone(GPIO7, 440U);
```

Trong EduFramework, `tone()` không có tham số thời lượng. Tín hiệu tiếp tục được duy trì cho tới khi tần số được thay đổi bằng một lần gọi `tone()` khác hoặc được dừng bằng `noTone()` [6]. Vì vậy, thời lượng của một tone trong bài lab được xác định bằng cách đặt `delay()` sau lời gọi `tone()`.

`tone()` sử dụng tầng PWM của framework để chuẩn bị đầu ra tương ứng, do đó chương trình không cần gọi `pinMode(GPIO7, OUTPUT)` trước khi phát tone [6].

4.2 noTone()

`noTone()` dừng tín hiệu đang được tạo trên Logical Pin được chỉ định [6].

noTone

Dừng tone trên PWM Logical Pin được chỉ định.

Cú pháp

```
noTone(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>pin</code>	PWM Logical Pin	Chân đang được sử dụng để phát tone.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
noTone(GPIO7);
```

Trong bài thực hành, API này được gọi sau tone cuối để tạo khoảng im lặng trước khi chuỗi được lặp lại.

4.3 Lưu ý về tài nguyên PWM

Các channel trong cùng một FTM của S32K1xx sử dụng chung một bộ đếm 16-bit [5]. Vì tần số đầu ra phụ thuộc vào bộ đếm chung này, các chức năng cần thiết lập tần số khác nhau không nên được sử dụng đồng thời trên những pin thuộc cùng một FTM.

Theo bảng ánh xạ PWM hiện tại của EduFramework, các Logical Pin được chia thành các nhóm sau [6]:

Nhóm PWM	Logical Pin
Group 1	LED_RED , LED_BLUE , LED_GREEN
Group 2	GPIO7 , GPIO8
Group 3	GPIO2 , GPIO3 , GPIO4 , GPIO5 , GPIO6

Bảng 7.4.1. Các nhóm Logical Pin chia sẻ tài nguyên PWM

Ghi chú

Không sử dụng `tone()` và `analogWrite()` đồng thời trên hai pin thuộc cùng một nhóm PWM. Ví dụ, `GPIO7` và `GPIO8` cùng thuộc Group 2. EduFramework hiện không thực hiện cơ chế khóa tài nguyên ở runtime, vì vậy việc lựa chọn pin phù hợp thuộc về ứng dụng.

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình sử dụng passive buzzer trên `GPIO7` để phát một chuỗi âm có cao độ tăng dần.

Chương trình cần đáp ứng các yêu cầu sau:

- Phát lần lượt bảy tone tương ứng với C, D, E, F, G, A và B.
- Duy trì mỗi tone trong 500 ms .

- Sau tone cuối, dừng buzzer trong `1000 ms` .
- Lặp lại toàn bộ chuỗi liên tục.

5.2 Chương trình

Trong `src/main.c` , triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```
#include "Arduino.h"

int main(void)
{
    setup();

    while (1)
    {
        tone(GPIO7, 262U); /* C */
        delay(500U);

        tone(GPIO7, 294U); /* D */
        delay(500U);

        tone(GPIO7, 330U); /* E */
        delay(500U);

        tone(GPIO7, 349U); /* F */
        delay(500U);

        tone(GPIO7, 392U); /* G */
        delay(500U);

        tone(GPIO7, 440U); /* A */
        delay(500U);

        tone(GPIO7, 494U); /* B */
        delay(500U);

        noTone(GPIO7);
        delay(1000U);
    }

    return 0;
}
```

Mã nguồn 7.5.1. Chương trình phát chuỗi tone bằng Passive Buzzer

Mỗi cặp lệnh gồm một giá trị tần số và một khoảng chờ `500 ms` . Khi chương trình chuyển từ giá trị tần số hiện tại sang giá trị lớn hơn, cao độ nghe được tăng theo quan hệ giữa frequency và pitch [1] .

Sau `494 Hz` , chương trình dừng tín hiệu trên buzzer và chờ `1000 ms` . Khoảng im lặng này tạo ranh giới rõ ràng giữa hai lần lặp của chuỗi. Khi vòng lặp bắt đầu lại, buzzer tiếp tục phát từ tần số đầu tiên.

Các giá trị tần số được viết trực tiếp trong chương trình để phần thực hành tập trung vào ảnh hưởng của tham số `frequency` . Cách đặt tên các tần số để làm mã nguồn dễ đọc hơn được giới thiệu trong phần Mở rộng.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board. Quan sát âm thanh phát ra từ passive buzzer và kiểm tra:

- Có bảy tone liên tiếp trong mỗi chuỗi.
- Cao độ tăng dần từ tone đầu tới tone cuối.

- Mỗi tone kéo dài khoảng 500 ms .
- Sau tone cuối có khoảng im lặng khoảng 1000 ms .
- Chuỗi được lặp lại liên tục.

KẾT QUẢ MONG ĐỢI

Passive buzzer phát bảy tone có cao độ tăng dần. Mỗi tone được duy trì khoảng 500 ms . Sau tone cuối, buzzer im lặng khoảng 1000 ms , sau đó chuỗi bắt đầu lại.

6 Mở rộng

Khi một chương trình sử dụng nhiều tone, việc viết trực tiếp các giá trị tần số ở mọi vị trí có thể làm mã nguồn khó đọc. Một cách đơn giản là đặt tên cho các giá trị thường dùng bằng hằng số.

6.1 Đặt tên tần số bằng hằng số

Ví dụ, tần số đầu tiên của bài thực hành có thể được định nghĩa:

```
#define NOTE_C4 262U
```

Khi đó lời gọi trong chương trình có thể viết:

```
tone(GPIO7, NOTE_C4);
```

Tên hằng số giúp thể hiện ý nghĩa của giá trị trong chương trình mà không làm thay đổi tần số được truyền cho API.

6.2 Bài tập mở rộng: tạo một giai điệu ngắn

Thay chuỗi tăng dần của bài thực hành bằng một giai điệu ngắn tự thiết kế.

Yêu cầu:

- Sử dụng ít nhất bốn tần số khác nhau.
- Sử dụng ít nhất hai thời lượng tone khác nhau.
- Có ít nhất một khoảng im lặng giữa các phần của giai điệu.
- Có thể định nghĩa các hằng số NOTE_... để mã nguồn dễ đọc hơn.
- Lặp lại giai điệu để có thể kiểm chứng nhiều lần trên phần cứng.

Không cung cấp lời giải cố định cho phần mở rộng. Tần số, thứ tự tone và thời lượng được lựa chọn theo giai điệu cần tạo.

KẾT QUẢ MONG ĐỢI

Passive buzzer phát được một giai điệu khác với chuỗi của bài thực hành chính, trong đó có thể nhận biết sự thay đổi về cao độ, thời lượng và các khoảng im lặng.

7 Tài liệu tham khảo

- [1] University of Minnesota, *Pitch is Frequency* , Introduction to Sensation and Perception.
- [2] University of Southern California, *EE 109 Unit H - Timers* , EE 109 - Introduction to Embedded Systems.
- [3] Augusta University, *PHYS 1111L Laboratory Manual* , PHYS 1111L, 2023.
- [4] Arduino, *Arduino Language Reference* , Arduino Documentation.
- [5] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.
- [6] EduFramework, *Tone, PWM and Logical Pin API* , EduFramework Source Code.

[7] MaaZEDU, *MaaZEDU Development Board Guide* .